EVMと日時実績ログを用いた個人タスク進捗モデルの提案と実装

明星大学情報学部情報学科
長研究室
22J5-114 野々村空河

2025年12月25日

目 次

1 はじめに **1**

2 研究の目的 **2**

3 関連研究 **2**

4 システムの実装 **5**

4.1 開発環境 5
4.2 システム構成 5
4.3 データモデル 6
4.4 主要機能の実装 8
4.4.1 タスク登録（TodoInput.jsx） 8
4.4.2 実績入力（ProgressEntry.jsx） 8
4.4.3 進捗指標計算（updateStats.js） 9
4.4.4 リスクレベル判定（SPI 基準） 9
4.4.5 当日の作業計画におけるタスク選定アルゴリズム 10
4.4.6 推奨作業時間（todayMinutes）の算出 10
4.4.7 当日計画の再計算処理 11
4.4.8 通知機能（Cloud Functions） 12
4.4.9 ログ編集機能（LogEditorModal） 13
4.4.10 データエクスポート機能 13
4.5 UI 画面の構成 13
4.5.1 カレンダー画面 14
4.5.2 タスク一覧画面 14
4.5.3 分析画面 14
4.5.4 ログ編集画面 16
4.5.5 設定画面 16
4.5.6 今日のプラン画面 17

5 評価・分析 **18**

5.1 評価の目的 18
5.2 評価対象および実験条件 19
5.3 分析指標および評価方法 20
5.3.1 分析 1：SPI に基づく遅延認知と行動変化 20
5.3.2 分析 2：EAC に基づく将来予測と行動変化 20
5.3.3 分析 3：今日のプラン遵守度と進捗指標（SPI・EAC）推移との整合性分析 21

6 結果・考察 **21**

6.1 分析 1：週間評価フェーズにおける SPI 改善効果の検証 21
6.1.1 使用データの概要 21
6.1.2 日次作業量および推定作業量の分布 22
6.1.3 SPI 算出に用いる指標の定義 22

6.1.4 分析 1：週ベース評価フェーズにおける SPI 改善効果の検証（結果） . . . 24
6.1.5 統計検定による SPI 変化の検証 25
6.1.6 $\mathrm{SPI}_{\mathrm{after}} \geq 1$ に到達したイベントの特徴 26
6.1.7 入力後に $SPI_{\mathrm{after}} \geq 1$ を満たす割合 27
6.2 分析 2：EAC に基づく将来予測と行動変化 28
6.2.1 分析の目的 28
6.2.2 使用データおよび遅延予測イベントの定義 28
6.2.3 遅延予測イベント前後の作業量変化 28
6.2.4 統計検定による前後差の検証 29
6.2.5 結果の解釈と考察 29

7 まとめ **30**

A 当日の作業計画におけるタスク選定アルゴリズム **31**
A.1 アルゴリズムの入力と出力 31
A.2 日次キャパシティの決定 31
A.3 候補タスク集合の定義 31
A.4 遅れ度および残日数の定義 32
A.5 優先度スコアの算出 32
A.6 日次キャパシティに基づく割当（`todayMinutes`） 32
A.7 フォールバック処理 33
A.8 疑似コード 33

1　はじめに

長期的な学習タスクや継続的なプロジェクトにおいて，日々の進捗を正確に把握し，適切なタイミングで行動を修正することは容易ではない。多くの既存の ToDo アプリはタスクの登録や締切管理を主眼として設計されており，進捗が計画からどの程度乖離しているのか，また現状の作業ペースで締切に間に合うのかといった情報を必ずしも十分に提供していない。特に，本研究で扱うような進捗効率や将来予測を，日次の実績に基づいて継続的に再計算し提示する機能は一般的ではない。その結果，利用者が遅延に気づくのは締切直前となり，必要な行動修正が後手に回るという問題が生じやすい。

このような課題は，利用者が自身の作業状況を把握し，計画との差を評価し，行動を修正するという一連の自己調整（Self-regulation）プロセスを継続的に回すことの難しさに起因すると考えられる。自己調整は，教育工学・学習科学分野において広く知られている概念であり，現状を把握する自己モニタリング，計画との乖離を評価する自己評価，および行動を修正する自己制御からなる循環過程として説明される。しかし，これらの過程を日常的なタスク管理の中で継続的に実践することは容易ではなく，従来のタスク管理ツールでは，この循環を十分に支援する情報設計がなされていない。

一方，プロジェクト管理分野では Earned Value Management（EVM）が広く用いられており，実績データに基づいて進捗を定量的に評価し，遅延の早期検知や完了見込みの予測を行う体系が確立されている。進捗効率を示す指標である SPI（Schedule Performance Index）や，現在の作業ペースを維持した場合の完了予測日を表す EAC（Estimate at Completion）は，計画と実績の乖離を直感的に把握する上で有用である。

しかし，従来の EVM は，計画値が比較的固定的であり，実績の更新頻度も限定的である企業プロジェクトを前提として設計されている。個人の学習タスクのように，作業量が日々変動し，実績が日次単位で蓄積される状況では，これらの指標をそのまま適用することは難しい。個人タスクにおいては，日次で更新される実績に基づき，進捗評価や将来予測を継続的に再計算し，適切なタイミングでフィードバックを行う仕組みが不可欠である。

また，進捗指標を単に可視化するだけでは，利用者がその情報を具体的な行動修正に結びつけることは難しい。進捗の遅延や将来リスクを「認知」し，それを基に行動を選択・調整できてはじめて，自己調整は機能すると考えられる。したがって，進捗評価指標は，行動変化と結びつく形で設計・提示される必要がある。

そこで本研究では，EVM の考え方を個人タスク管理向けに再構成し，日次実績ログに基づいて進捗指標を自動的に再計算する個人タスク進捗管理モデルを提案する。本モデルでは，直近の実績ペースを表す pace7d（短期的な行動傾向を反映する指標），進捗効率を示す SPI，および将来的な完了見込みを示す EAC を用い，SPI による遅延状態の認知，EAC による将来リスクの評価，さらにそれらに基づき，日単位で実行可能な行動へ落とし込む機構として「今日のプラン」を提示することで，利用者の自己調整プロセス全体を支援することを目指す。

本研究は，進捗指標の精度そのものを評価することを主目的とするのではなく，実利用ログに基づき，これらの指標やプラン提示が，利用者の遅延認知や行動選択にどのような影響を与え，行動修正につながり得るかに着目する点に特徴がある。本論文では，提案モデルを実装した進捗管理アプリを開発し，実際の利用ログを用いた分析を通じて，遅延の早期検知，行動修正，および進捗理解の促進という観点から，提案手法の有効性を検証する。

2　研究の目的

本研究の目的は，個人の長期タスクに対して「遅延の早期検知」「行動修正」「進捗理解の促進」を実現する進捗管理モデルを構築することである。本モデルは，日次実績ログに基づく定量的な進捗評価を通じて，利用者が自身の作業状況や将来リスクを早期に把握し，行動修正を行えるよう支援することを目的とする。

第一章で述べたように，従来の ToDo アプリはタスクの登録や締切管理に重点が置かれており，計画と実績の乖離や将来の締切リスクを日常的に判断するための情報が十分に提供されていない。また，従来の EVM は固定的な計画値を前提としており，作業計画が日々変動する個人の長期タスクへそのまま適用することは難しい。

本研究では，このような課題に対し，固定的な計画値に基づく進捗管理ではなく，日次実績ログから算出される動的な指標を用いることで，個人タスクに適した進捗評価と将来予測を行う。具体的には，実績ペースや必要ペースに基づく進捗効率指標（SPI）と，予測完了日（EAC）を継続的に更新することで，計画と実績の乖離を定量的に把握し，行動修正につなげることを目指す。ただし，本研究は作業時間の見積精度そのものの向上を目的とはせず，乖離が生じることを前提とした支援に主眼を置く。

以上の目的を達成するため，本研究では以下の点を具体的な研究目標として設定する。特に，自己調整（Self-Regulation）プロセスの主要な要素である自己モニタリング，自己評価，自己制御の各段階を支援するため，技術的目標と理論的目標を以下に定める。

- 個人タスクにおける進捗状態を定量化し，遅延リスクを早期に認知可能な進捗評価モデルを構築する。
- 進捗評価結果を行動判断に結びつけるためのフィードバック機構を設計し，利用者の行動修正を支援する。
- 将来の完了見込みを継続的に可視化することで，行動と進捗予測の関係を理解しやすくする。
- 自己調整（Self-Regulation）理論に基づき，自己モニタリング・自己評価・自己制御の循環を継続的に支援する進捗管理モデルの理論的基盤を確立する。

本研究は，EVM 指標を日次実績ログと組み合わせて動的に運用し，自己調整理論に基づくフィードバックを提供する点に特徴を有する。この理論的立場および目的設定は，第 4 章で述べるシステム設計と，第 5 章で行う評価・分析の前提となる。

3　関連研究

近年，個人向けのタスク管理サービスは数多く提供されており，Google ToDo，TickTick，Notion，Trello などが広く利用されている。これらのサービスは，タスクの登録，締切設定，通知といった基本機能を通じて，ユーザーの作業整理や忘却防止を支援している。

一方で，多くのサービスは，進捗の定量的な把握や，見積もりと実績の比較に基づく支援までは十分に踏み込んでいない。たとえば TickTick はポモドーロタイマーによる作業ログ記録を提供しているが，見積時間 E と実績時間 A の体系的な比較や，進捗効率指数（SPI）や予測完了日（EAC）といった定量指標を用いたフィードバックには対応していない。また，Notion や Trello

のようなデータベース型サービスでは柔軟なタスク管理が可能であるものの，進捗の自動評価やペース分析といった機能はユーザー自身の設計に委ねられている。

表 1 に，代表的なタスク管理サービスと進捗管理機能の有無を比較した結果を示す。なお，本研究システムについても未実装機能が存在するため，比較が特定のシステムに過度に有利とならないよう留意している。

表 1: 主要タスク管理サービスと進捗管理機能の比較

機能項目	Google ToDo	TickTick	Notion	Trello	本研究
タスク登録・締切設定	○	○	○	○	○
見積時間（E）の設定	×	△	△	×	○
実績時間（A）の記録	×	○	△	×	○
SPI/EAC の表示	×	×	×	×	○
pace7d に基づく評価	×	×	×	×	○
作業提案通知	×	△	×	×	△
コラボレーション	×	○	○	○	×
スマホ UI 対応	○	○	○	○	△（PC 特化）

学習者の自己調整と進捗把握の困難性

学習科学の分野では，学習者が自身の進捗や作業量を正確に把握することの難しさが繰り返し指摘されている。野上ら [1] は，学習者が自らの学習量を過大または過小に評価する傾向を報告しており，主観的な自己評価のみに基づく学習管理には限界があることを示している。

一方で，学習時間や作業ログといった客観的な行動データを可視化することは，学習行動の調整（self-regulation）を促進する要因となることが知られている。これらの知見は，行動データに基づく定量的なフィードバックが，学習や作業の自己管理において重要な役割を果たすことを示唆している。

自己調整学習モデルと行動フィードバック

自己調整（Self-regulation）は，学習者や作業者が目標達成に向けて自身の行動を調整する過程として，教育工学および心理学の分野で広く研究されてきた。Kanfer (1990) は，セルフモニタリング（自己観察）とフィードバックの重要性を指摘し，行動データの継続的な記録と評価が目標達成能力の向上に寄与することを示している。

これらの研究では，行動の記録（モニタリング），進捗や差異の評価，それに基づく行動修正が循環的に行われることが重要であるとされている。このような枠組みは，進捗管理システムを設計する上での理論的背景として位置づけられる。

作業時間見積もりの誤差に関する研究

作業時間の見積もりに関しては，Connolly and Dean (1997) がソフトウェア開発を対象に，見積時間が体系的に過小評価される傾向を報告している。この背景には，楽観バイアスや過度な自信といった認知的要因が存在することが指摘されている。

これらの研究から，作業時間の見積もりには一定の誤差が生じやすく，見積値と実績値の乖離を前提とした進捗管理の必要性が示唆される。

EVM の個人タスクへの応用と課題

Earned Value Management（EVM）は，プロジェクト管理において計画と実績の差異を定量的に評価する手法として広く用いられており，SPI（Schedule Performance Index）や EAC（Estimate at Completion）といった指標が知られている。

しかし，従来の EVM は企業プロジェクトを主な対象として設計されており，個人の学習タスクのように日々の実績が大きく変動する文脈では，そのまま適用することが難しい。具体的には，

- 計画値（PV）が柔軟な個人計画に適合しづらい
- 実績ログが日単位で更新されるため，指標の再計算頻度が高い
- 完了見込みの変化が短期間で大きく変動しうる

といった課題が挙げられる。

実務においては，PMI や NASA，米国国防総省（DoD）などの EVM ガイドライン [3] において，$\mathrm{SPI} \geq 1.0$ を計画通り，$0.8 \leq \mathrm{SPI} < 1.0$ を軽度の遅延，$\mathrm{SPI} < 0.8$ を重大な遅延とする基準が用いられる例が多い。これらの基準は，進捗リスクを定量的に把握する指標として広く参照されている。

Self-regulation 理論と進捗管理システム

教育工学・心理学の分野では，自己調整（Self-regulation）は，以下の 3 段階からなる循環的プロセスとして整理されることが多い。

- Self-monitoring：自身の行動や進捗を把握する段階
- Self-evaluation：計画や目標との差を評価する段階
- Self-control / adjustment：評価結果に基づき行動を修正する段階

表 2 は，これらの各段階と，進捗管理システムにおいて想定される機能との対応関係を整理したものである。

表 2: Self-regulation 段階と進捗管理システム機能の対応

Self-regulation 段階	内容	想定される機能例
Self-monitoring	現状把握	作業ログ記録
Self-evaluation	評価・将来予測	進捗指標・予測表示
Self-control / adjustment	行動修正	作業計画の見直し

関連研究の整理

以上の関連研究の整理から，既存のタスク管理サービスは，タスクの登録や締切管理を主眼としており，進捗の定量評価や将来予測に基づく行動修正を一体的に支援する仕組みは限定的であることが分かる。また，EVM に基づく進捗管理手法は，主に企業プロジェクトを対象として設計されており，固定的な計画値を前提とする点で，個人の長期タスクへの適用には課題が残る。さらに，自己調整理論は学習支援において有効性が示されているものの，日次実績ログに基づく進捗指標と結び付けて実装・評価した研究は多くない。

4　システムの実装

本章では、本研究で開発した進捗マネジメントアプリの構成と実装について述べる。まず開発環境を示し、次にシステム全体の構造、データモデル、主要機能、UI 設計について順に説明する。

4.1　開発環境

本研究のアプリケーションは，Web ブラウザ上で動作するシングルページアプリケーション（SPA）として実装した。利用者が日次で実績を入力し，その結果に応じて進捗指標を即座に再計算・提示する必要があることから，リアルタイム性と保守性を重視した構成を採用している。

主な使用技術は以下のとおりである。

- フロントエンド：React（Vite）
- 言語：JavaScript / JSX
- バックエンド：Firebase（Authentication, Firestore, Cloud Functions）
- 通知：Firebase Cloud Messaging（FCM）
- 実行環境：Node.js
- バージョン管理：Git / GitHub

Firebase は，認証，データベース，バックエンド処理（Cloud Functions），通知（FCM）を統合的に提供しており，日次実績ログの更新を契機とした進捗指標の自動再計算や，利用者入力に即応したフィードバック処理を実装しやすい点に利点がある。このため，本研究で提案する「実績入力 → 評価 → 行動支援」という進捗管理モデルの実装基盤として適していると判断し，採用した。

4.2　システム構成

本システムは、React を用いたフロントエンド、Firestore を中心とするデータベース層、そして Google Cloud Functions を用いたバックエンド処理の 3 層構成で動作する。図 1 に全体構成を示す。

ユーザー操作はフロントエンドで行われ、データは Firestore に保存される。Firestore の更新をトリガとして Cloud Functions が進捗指標の計算や通知スケジューリングを自動実行することで、ユーザーの入力に応じて常に最新状態の進捗データが維持される。

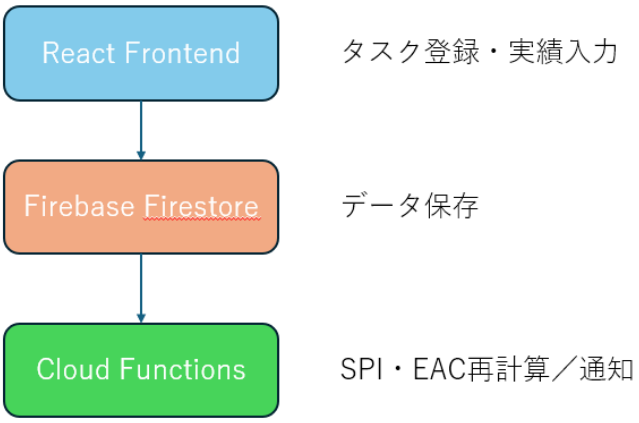

図 1: 本研究システムの全体構成

フロントエンドでは、タスク登録や実績入力、分析画面での可視化といったユーザーとのインタラクションが中心となる。タスク一覧画面には、EVM 指標（SPI・EAC・残作業量など）がリアルタイムに反映され、各タスクの遅延リスクは SPI や EAC に基づいて「良好」「注意」「遅延」に色分けされて表示される。

一方、Firestore ではタスクデータ・日次実績ログ・必要ペース（requiredPace）・SPI・EAC といった EVM 指標を保持する。日次ログは actualLogs として日付キーで管理され、後述する pace7d の計算に利用される。

Cloud Functions では Firestore の変更を監視し、以下の主要処理を自動実行する。

- updateStats.js：日次実績ログが更新されるたびに、pace7d、残作業量、必要ペース、SPI、EAC を再計算し Firestore に書き戻す。ログ編集にも対応しており、過去の実績が修正された場合でも全期間の SPI および EAC を整合的に再計算する。
- scheduleMorningSummary.js：毎朝、タスクの優先度や遅延リスクを SPI に基づいて評価し、当日の推奨タスクを通知する。
- scheduleProgressReminder.js：夜間に実績未入力を検知し、記録を促すリマインド通知を送信する。

この 3 層構成により、ユーザーが実績入力を行うだけで EVM 指標の自動更新・通知処理・可視化が連鎖的に行われ、進捗管理の循環（記録 → 評価 → フィードバック）が自動的に機能するアプリケーション構造が実現されている。

4.3 データモデル

本研究のアプリケーションは，タスク情報・日次実績ログ・EVM 指標を総合的に管理するために，Firestore を用いた階層型データモデルを採用している。図 2 に，本アプリで用いる主要データ構造の概要を示す。本モデルは，ユーザーが実績を入力するたびに Cloud Functions が自動的に再計算を行い，最新の進捗指標が常に Firestore 上に反映されることを前提として設計している。

本研究のアプリでは，進捗管理に必要な情報を以下のように保持する。

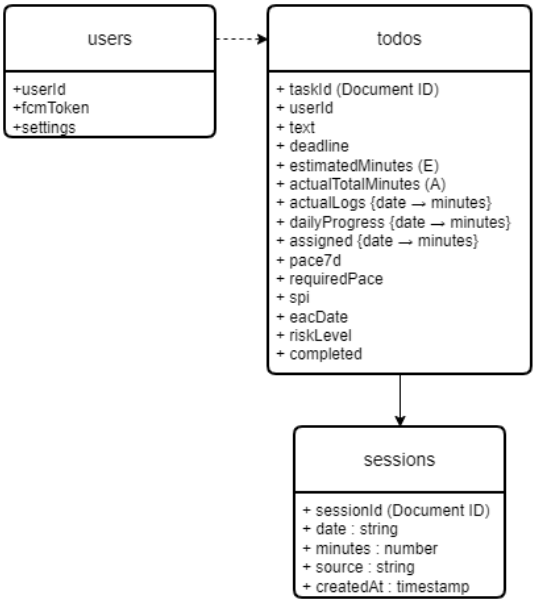

図 2: Firestore のデータモデル（タスク・実績ログ）

- **todos コレクション（タスク）**
  - text：タスク名
  - estimatedMinutes（E）：見積時間
  - actualTotalMinutes（A）：累積実績
  - actualLogs：日次実績ログ（YYYY-MM-DD: 分）
  - pace7d：直近 7 日間の平均実績ペース
  - requiredPaceAdj：期日までに完了するための必要ペース
  - spi：進捗効率指数（SPI）
  - eacDate：予測完了日（EAC）
  - deadline：締切日時
  - completed：完了フラグ
  - createdAt：タスク登録日時
  - lastProgressAt：最後に実績が記録された日時
  - userId：タスク所有ユーザ ID
- **sessions サブコレクション**
  - date：実績入力日
  - minutes：入力された作業時間
  - source：入力経路（手動 / 通知から）
  - createdAt：記録時刻

表 3: todos コレクションの主なフィールド（改訂版）

フィールド名	概要
`text`	タスク内容（例：IT パスポート 100 時間勉強する）。
`deadline`	タスク全体の締切日時 D。
`estimatedMinutes`	見積総学習時間 E［分］。進捗率・残作業量算出の基礎となる。
`actualLogs`	日付ごとの実績時間 $A(t)$ を保存するマップ。pace7d，EAC，SPI の再計算に使用。
`actualTotalMinutes`	累積実績時間 $\sum_t A(t)$［分］。
`dailyProgress`	日別の累積進捗率（その日までの EV ／ 総 EV）。
`actualProgress`	現時点の累積進捗率（EV ／ 総 EV）。
`idealProgress`	線形計画に基づく理想進捗率。計画との差分（遅れ判定）に利用。
`pace7d`	直近 7 日間の平均学習ペース［分／日］。SPI・EAC の基礎指標。
`requiredPace`	遅れを考慮しない基本の必要ペース（残作業量 ÷ 残日数）。
`requiredPaceAdj`	遅れ状況（idealProgress と actualProgress の差）を考慮し，遅れ量を残日数で均等に取り戻す前提で算出される調整後必要ペース［分／日］。
`spi`	SPI（進捗効率指数：pace7d ／ requiredPaceAdj）。
`spi7d`	直近 7 日間の SPI。pace7d と requiredPaceAdj から算出される。
`eacDate`	現在のペースを続けた場合の予測完了日（EAC）。
`riskLevel`	遅延リスクのカテゴリ（“ok”, “warn”, “danger” など）。
`completed`	タスク完了フラグ。
`createdAt`	タスク登録日時。
`lastProgressAt`	最後に実績が記録された日時。
`userId`	タスク所有ユーザ ID。
`eacHistory`	日次で再計算された EAC の履歴を保持し，進捗推移分析に利用する。

- **users コレクション**
  - fcmToken：通知送信に使用するトークン
  - settings：通知 ON/OFF などの設定
  - loginCount：利用回数（継続性の指標）

特に actualLogs と pace7d は本研究特有のデータ構造であり，EVM 指標を日次ベースで再計算するための基盤となっている。

4.4 主要機能の実装

本節では，本研究で開発した進捗マネジメントアプリにおける主要機能について述べる。以下では，タスク登録，実績入力，進捗指標の更新，実績ログの編集，およびデータのエクスポートといった処理を中心に，各機能がどのように連携して実装されているかを説明する。

4.4.1 タスク登録（TodoInput.jsx）

ユーザーがタスク名，締切日時，見積時間（E）を登録すると，Firestore の `todos` コレクションに新規ドキュメントが作成される。登録された見積時間 E は，後続の進捗指標計算処理において参照され，残作業量や必要ペース，SPI，EAC などの各指標の算出に利用される。また，タスクはラベルや作成日時，ユーザー ID とともに保存され，後述する分析機能に利用される。

4.4.2 実績入力（ProgressEntry.jsx）

ユーザーが 1 日の作業時間を入力すると，以下の処理が自動で行われる。

1. `actualLogs[日付]` への実績加算
2. `actualTotalMinutes`（累積実績 A）の更新
3. `sessions` サブコレクションへの入力ログ保存

記録された日次実績は，pace7d（直近 7 日平均）の算出に利用され，SPI や EAC などの進捗指標の再計算に反映される。また，実績は後から修正可能であり，過去ログの変更にも対応している。

4.4.3 進捗指標計算（updateStats.js）

Firestore の `actualLogs` が変更されるたびに，Cloud Functions により進捗指標の再計算処理が実行される。本研究では，日次実績ログに基づき，タスクの進捗状態を継続的に更新するため，updateStats.js に指標計算処理を集約した。本機能により，ユーザーの実績入力や過去ログの修正に応じて，Firestore 上の指標値が常に最新状態となるよう維持される。

本研究で updateStats.js が再計算対象とする主な指標を以下に示す。

- 残作業量：$R = E - A$
- 直近平均ペース：pace7d（過去 7 日間の平均作業時間）
- 必要ペース：requiredPaceAdj（締切までに必要な 1 日あたり作業量）
- 進捗効率指数：$\mathrm{SPI} = \frac{\mathrm{pace7d}}{\mathrm{requiredPaceAdj}}$
- 予測完了日（EAC）：現在の pace7d を継続した場合の完了見込み日

計算後，Cloud Functions は各指標を Firestore の該当フィールドに書き戻し，フロントエンドの表示および後続処理で参照可能な状態とする。

EAC 予測日の履歴更新　本研究では EAC を最新値として保持するだけでなく，日次の実績ペースに応じて変化する予測完了日を履歴として保存するため，`eacHistory` を導入した。updateStats.js は再計算の実行時に，当日付の EAC を `eacHistory[YYYY-MM-DD]` として記録することで，予測完了日の変化を時系列データとして蓄積する。

また，実績ログの編集により過去の `actualLogs` が変更された場合には，全期間の pace7d, requiredPaceAdj，SPI，EAC を再計算し，`eacHistory` を含む関連フィールドを整合的に更新するように設計している。

4.4.4 リスクレベル判定（SPI 基準）

本システムでは，進捗指標として算出される SPI の値に基づき，タスクの進捗状態を段階的に分類する。SPI の値は Cloud Functions 内の判定ロジックで参照され，以下の 3 段階のリスクレベルとして整理される。

- $\mathrm{SPI} \geq 1.0$：良好
- $0.8 \leq \mathrm{SPI} < 1.0$：注意

- SPI < 0.8：遅延

また，EAC が締切日時を超過すると判定された場合や，締切日時を過ぎてもタスクが完了していない場合には，SPI の値に関わらず遅延状態として扱う。これらの判定結果は `riskLevel` として Firestore に保持され，後述するユーザーインタフェース層において参照される。

4.4.5 当日の作業計画におけるタスク選定アルゴリズム

本節では,「今日のプラン」機能において，当日に取り組むタスクをどのように選定しているかについて概要を述べる。本機能は，利用者が当日に優先的に着手すべきタスクを把握できるようにすることを目的として，進捗指標および締切情報をもとに，当日の作業候補を最大 3 件提示するものである。

タスク選定処理は，フロントエンドおよびバックエンドの双方から利用可能な共通モジュール（`shared/planEngine.js`）として実装されており，実行環境に依存しない一貫した判定結果が得られる構成としている。

アルゴリズムでは，まず未完了であり締切日時が設定されているタスクを候補として抽出する。この際，遅延状態にあるタスクのみを対象とするのではなく，締切が近づいている進捗良好なタスクも候補に含めることで，締切直前の未着手を防ぐことを意図している。

候補タスクは，進捗状況，締切までの残期間，および残作業量といった情報を参照して評価され，優先度順に整列される。その結果にもとづき，上位のタスクを当日の作業計画として提示する。

なお，本節ではタスク選定アルゴリズムの詳細な数式や判定ロジックについては述べず，実装上の位置づけと処理の流れに焦点を当てる。アルゴリズムの具体的な定義および擬似コードについては，付録 A において詳述する。

4.4.6 推奨作業時間（todayMinutes）の算出

選定された最大 3 件のタスクに対し，本節では当日に割り当てる推奨作業時間 `todayMinutes` をどのように算出するかを述べる。本システムでは，単に必要ペースを提示するのではなく,「最低限必要な作業量」「遅延回復を考慮した作業量」「余剰キャパシティの活用」という三段階の考え方に基づいて，日次作業時間を決定する。

(1) 日次キャパシティの確定　当日の作業キャパシティを C［分］とする。基本的にはユーザー設定の `dailyCap` を用いるが，未設定，0 以下，または不正値（Infinity 等）の場合は，安全策として既定値 120 分にフォールバックする。

再計算時（初期計算以外）においては，既に当日消費された作業時間を考慮し，残キャパシティを上限として以降の割当てを行う。

(2) 第 1 パス：最低ライン（required）の割当て　各タスク i について，まず「締切までに間に合わせるために最低限必要な 1 日あたり作業量」を最低ラインとして割り当てる。

残作業量を

$$R_i = E_i - A_i$$

とし，必要ペース $required_i$ は，`requiredPaceAdj` が存在する場合はそれを，存在しない場合は `requiredPace` を用いる。

このとき，各タスクへの最低割当て量 $minMinutes_i$ は，

$$minMinutes_i = \min(required_i,\ R_i)$$

とし，残キャパシティの範囲内で割当てを行う。割当て後は残キャパシティを更新する。

(3) 第 2 パス：回復ラインの割当て　次に，各タスクについて，当日の作業によって進捗効率の改善を目標とした回復ラインを考慮する。

この回復目標量を $recoverMinutes_i$ とし，第 1 パスで割り当てた量との差分が存在する場合，残キャパシティの範囲内で追加割当てを行う。

$$add_i = \min(recoverMinutes_i - minMinutes_i,\ C_{\text{remain}})$$

これにより，遅延状態にあるタスクについては，単なる締切遵守にとどまらず，進捗状態の改善を考慮した作業量が提示される。

(4) 第 3 パス：余剰キャパシティの配分　第 2 パス終了後もキャパシティが残っている場合，締切が近い順にタスクを並べ替え，1 タスクあたり最大 30 分を上限として追加割当てを行う。

この処理は，余剰時間を前倒し的に活用するための補助的な割当てとして位置づけられる。

(5) フォールバック処理　上記 3 パスを経ても割当てが 1 件も成立しない場合に限り，フォールバック処理を行う。この場合，候補集合の範囲内で最大 3 件を対象とし，締切までの残日数 D_i に基づく日割り負荷を割り当てる。

具体的には，

$$todayMinutes_i = \left\lceil \frac{R_i}{D_i} \right\rceil$$

（5 分単位で切り上げ）とする。

本処理は,「当日の作業計画が空になる」状態を回避するための最小限の補助的処理として位置づけている。

(6) 小結　以上のように，`todayMinutes` の算出は，最低限の達成ライン，進捗回復の考慮，および余剰キャパシティの活用という三段階の構成に基づいて行われる。これにより，利用者の当日の作業可能時間と進捗状態を反映した，実行可能な作業計画を提示することを目的としている。

4.4.7　当日計画の再計算処理

本節では,「今日のプラン」に対して再計算が行われる条件と，その際の処理内容について述べる。

本システムでは，ユーザーが当日の作業実績を入力した場合や,「今日のプランを更新」操作を行った場合に，当日計画の再計算が実行される。再計算は新たな最適化アルゴリズムを導入するものではなく，前節までに述べたタスク選定および作業時間割当アルゴリズムを同一条件で再適用する処理として位置づけられる。

(1) 再計算のトリガ　当日計画の再計算は，以下のタイミングで実行される。

- 当日の作業実績（actualMinutes）が入力・更新された場合
- ユーザーが手動で「今日のプランを更新」操作を行った場合

これにより，当日残り時間や進捗状況の変化を即座に計画へ反映できる。

(2) 再計算時の処理内容　再計算時には，前節までに示したタスク選定アルゴリズム（4.4.5）および推奨作業時間算出アルゴリズム（4.4.6）を，当日の最新状態にもとづいて再実行する。
具体的には，

- 未完了タスクを対象として候補集合を再生成する
- スコア計算にもとづき最大 3 件のタスクを再選定する
- 当日残キャパシティを考慮した 3 パス方式（最低ライン・回復ライン・前倒し割当）により todayMinutes を再算出する

この際，すでに当日実施された作業時間は消費済みキャパシティとして扱われ，再計算後の計画は「当日残り時間に対する作業計画」として提示される。

(3) 再計算処理の位置づけ　本研究における再計算処理は，計画を頻繁に入れ替えること自体を目的とするものではなく，

- 作業実績の入力によって進捗認知を促すこと
- 進捗の変化に応じて，その日の残り行動を調整可能にすること

を目的としている。
すなわち，再計算は「当初計画の失敗を修正する最適化処理」ではなく，利用者の行動結果を反映した**動的な行動支援機構**として位置づけられる。

(4) 小結　以上より，本システムの当日計画再計算は，タスク選定および時間割当のアルゴリズムを同一の設計思想のもとで再適用する処理であり，計画提示と実績入力を往復することで，利用者の自己調整行動を支援する構造となっている。

4.4.8　通知機能（Cloud Functions）

本システムでは，ユーザの自己調整的な学習・作業習慣を支援するために，Cloud Functions と Firebase Cloud Messaging（FCM）を用いた通知機能を実装している。単なる時刻ベースのリマインダではなく，EVM 指標（requiredPaceAdj，SPI，EAC，riskLevel）に基づき進捗状況に応じたフィードバックを提供する点に特徴がある。通知は主に次の 3 種類である。

(1) 朝のプラン通知　毎朝所定時刻に Cloud Functions がユーザデータを読み込み，当日の実績・残作業量・必要ペースを反映した作業計画（todayMinutes・優先タスク集合）を生成する。そのうえで FCM によりユーザ端末へ通知を送信し，アプリ内の「今日のプラン」画面を確認するよう促す。

本実装では，

「今日のプランを確認してみましょう。」

といった簡潔なメッセージのみを送る設計とした。

(2) 夜の実績入力リマインド　当日の実績が未入力の場合には，夜間に入力を促す通知を送信する。EVM における各指標（pace7d，SPI，EAC）は日次の実績ログを前提としており，入力漏れが続くと精度が低下するため，本リマインド通知は進捗管理モデル全体の信頼性を支える役割を持つ。

(3) 通知生成とデータ更新の関係　実績入力や過去ログ編集が行われるたびに，Cloud Functions により requiredPaceAdj，SPI，EAC などの指標が再計算される。通知はこれら最新の指標に基づいて生成されるため，ユーザは現在の進捗状況に応じたフィードバックを受け取ることができる。一方で，過度な通知頻度を避けるため，プラン再計算（4.4.7）を行っても即時通知は行わず，次回の定期通知（朝または夜）で反映される設計としている。

(4) 本機能の位置づけ　通知機能は，ユーザが自分でタスクを選び，計画し，記録し，振り返るという自己調整（Self-Regulated Learning）のサイクルを支援する役割を担う。特に朝のプラン通知と遅延リスク通知は,「現状把握 → 乖離の認識 → 行動修正」という流れをデータに基づいて促す点で，本研究の進捗管理モデルを構成する重要な要素となっている。

4.4.9　ログ編集機能（LogEditorModal）

ユーザーは過去の実績を修正できる。修正後は Cloud Functions が呼び出され，pace7d，requiredPaceAdj，SPI，EAC，riskLevel が再計算される。本研究では，履歴を遡って修正しても指標の整合性が維持されるように，すべての計算を日次ベースで再実行する設計とした。

4.4.10　データエクスポート機能

分析や評価実験のため，記録された実績ログや EVM 指標を CSV 形式でエクスポートできる機能を実装した。これにより，通知反応率や見積誤差の推移など，研究用データを外部で解析することが可能になる。

4.5　UI 画面の構成

本節では，ユーザーが実際に操作する UI 画面の構成について述べる。4.4.8 節で説明した各種処理（EVM 指標の計算，リスク判定，通知処理など）はバックエンドで自動的に行われ，本節ではそれらの結果がどのように画面上で提示され，ユーザーの自己調整行動を支援しているかに焦点を当てる。

本アプリの主要な画面は，カレンダー画面，タスク一覧画面，分析画面，ログ編集画面，設定画面の 5 つである。

本節では，主要な UI を図 3 〜 図 9 に示し，各画面の役割と研究上の意義を述べる。

4.5.1　カレンダー画面

カレンダー画面（図 3）では，月単位でタスクの締切日や実績入力の有無を俯瞰できる。各日付には締切を持つタスクが表示され，締切が集中している期間や，全く作業していない期間が一目で確認できる。

この画面は，ユーザーが長期的な作業スケジュールを把握し,「どの時期に負荷が偏っているか」「どのあたりから着手すべきか」といった判断を行うための手がかりを提供する役割を持つ。

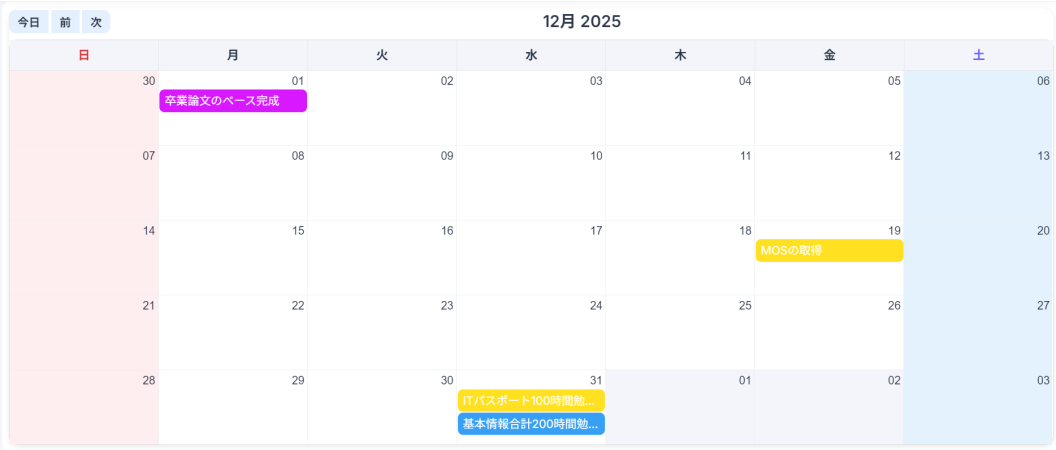

図 3: カレンダー画面（月間のタスク進捗と締切の俯瞰）

4.5.2　タスク一覧画面

タスク一覧画面（図 4）は，本アプリの中心的な画面であり，各タスクの概要と進捗状況が一覧で表示される。各カードには，タスク名，締切，見積時間，累積実績，残作業量に加えて，最新の EVM 指標（pace7d，必要ペース，SPI，EAC）が要約された形で示される。

特に，SPI に基づくリスクレベルは，カードの背景色やバッジとして視覚的に表現される。SPI が 1.0 以上のタスクは緑色（良好），0.8 以上 1.0 未満は黄色（注意），0.8 未満は赤色（遅延）として表示されるため，ユーザーは「どのタスクが危険域に入りつつあるか」を瞬時に把握できる。これにより，着手順序の決定や当日の作業計画に反映しやすくなる。

4.5.3　分析画面

分析画面（図 5）では，EVM 指標を用いた時系列の可視化が行われる。主な表示内容は次のとおりである。

- 日別の作業時間と累積作業量

図 4: タスク一覧画面（SPI, EAC, 残量, 必要ペースなどのリアルタイム可視化）

- 直近 7 日間および 30 日間の平均ペース
- タスク別の SPI 推移（折れ線グラフ）
- 理想的な進捗曲線と実績の比較

これらのグラフにより，ユーザーは「いつからペースが落ち始めたのか」「行動の変化が SPI や EAC にどのように影響したか」を視覚的に理解できる。特に SPI の折れ線グラフは，注意状態や遅延状態に入ったタイミングを振り返る材料となり，自身の学習・作業習慣をメタ認知するきっかけを与える。

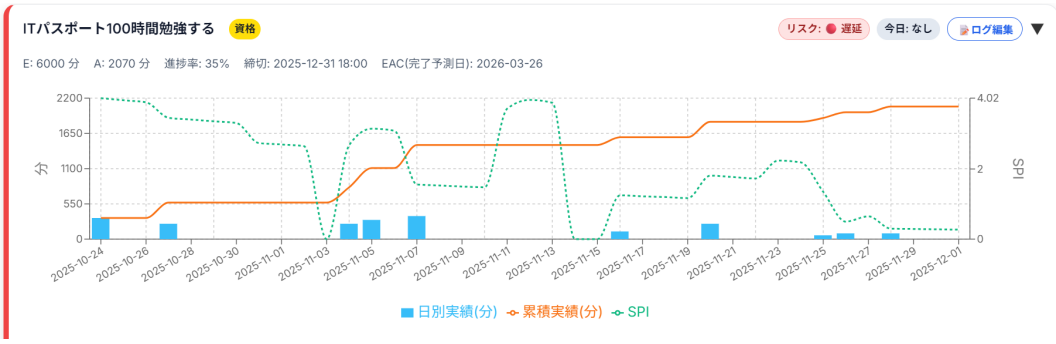

図 5: 分析画面（作業時間推移，直近平均，SPI 推移）

さらに本研究では，タスクごとに日次で再計算される EAC（予測完了日）の履歴を折れ線グラフとして可視化している。このグラフでは，縦軸を 2 つ用い，左軸に残作業量，右軸に予測完了日（EAC）を配置することで,「残作業量の増減」と「予測完了日の遅れ・回復」を同時に把握できるようにした。

これにより，ユーザーは，

- どの時点で EAC が締切を超過し始めたか

- 作業量の増減が EAC にどのように影響したか
- 行動改善（作業ペース向上）が EAC をどの程度回復させたか

を視覚的に確認できる。

図 6 に，本研究で導入した EAC（予測完了日）の日次推移を可視化した折れ線グラフを示す。残作業量（左軸）と EAC 予測日（右軸）を同時に表示することで，実績ペースの変化が締切遅延にどの程度影響しているかを直感的に確認できる。

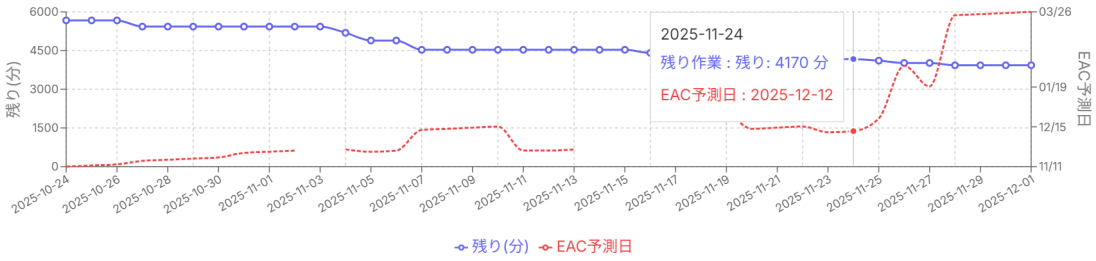

図 6: EAC 予測日の時系列グラフ（残作業量との比較）

一般的なタスク管理では締切遅延の兆候を把握しづらいが、EAC の時系列変化を見ることで「どの行動が遅れの原因となったか」「改善行動がどの程度効いたか」を直接確認でき、これは自己調整学習におけるメタ認知的フィードバックとして重要な役割を果たす。

4.5.4 ログ編集画面

ログ編集画面（図 7）では，日付ごとに記録された実績時間を一覧し，誤入力や記録漏れを修正できる。ユーザーはカレンダーや一覧から日付を選択し，当日の作業時間を増減させることができる。

この画面は，実績データの欠損や誤差を補正し，より現実に近い進捗記録を維持するためのものである。ユーザーにとっては,「ちゃんと記録できていない」という心理的なストレスを軽減し，日次記録の継続を支える役割も持つ。

4.5.5 設定画面

設定画面では，図 8 に示す画面で通知時刻や通知のオン・オフを設定でき、タスクラベルの管理，アプリのバージョン情報などを確認・変更できる。通知時刻を生活リズムに合わせて調整できるようにすることで，ユーザーは無理のないタイミングでリマインドを受け取ることができる。

また，ログイン回数やバージョン情報（コミット ID）を表示することで，利用継続の度合いやシステム更新履歴を簡単に把握できるようにしている。これらは研究上は補助的な情報であるが，ユーザーの利用意欲や再現性の確保に寄与する。

以上のように，本アプリの UI は，4.4 節で述べた EVM 指標計算やリスク判定の結果を，ユーザーが理解しやすい形で提示することを目的として設計されている。各画面はそれぞれ異なる時間スケール（1 日，1 週間，1 か月）や視点（タスク単位，全体の傾向）からフィードバックを与えることで，ユーザーの自己調整を多面的に支援している。

図 7: ログ編集画面（入力忘れや誤入力の修正に対応）

- アプリの通知をユーザーの生活リズムに適応
- ラベル管理によりカテゴリ別学習の分析が可能
- バージョン情報により再現性と信頼性を担保
- ログイン回数の可視化は継続利用の動機づけになる

4.5.6 今日のプラン画面

今日のプラン画面では，4.4 節で述べたタスク選定アルゴリズムに基づいて，当日に取り組むべき最大 3 件のタスクが提示される。各タスクカードには

- 残作業量 R
- 必要ペース（requiredPaceAdj）
- 当日割り当てられた推奨作業量（todayMinutes）

が表示され，ユーザーは優先すべきタスクと作業量を直感的に把握できる。

本画面には「今日のプランを更新」ボタンが配置されており，ユーザーが任意のタイミングで当日のプランを再計算できる。再計算時のアルゴリズムは 4.4.7 節で述べたとおり，当日これまでの実績時間（actualToday）を考慮し，残キャパシティに応じた推奨作業量が再算出される。

さらに，本画面では当日の計画値（Planned）と実績値（Actual）が棒グラフによって可視化される。これによりユーザーは，その日の計画達成度を視覚的に確認でき，日中の自己調整行動を支援する役割を果たす。

図 8: 通知設定画面

図 9: 今日のプラン画面（タスク一覧と割当作業量の提示）

5　評価・分析

5.1　評価の目的

本分析では，第 4 章で述べた pace7d,SPI,EAC を用い，利用者が進捗の遅延状態を認知した際に行動変化が生じるかを評価する。

自己調整学習は一般に，(1) 現状を把握する自己モニタリング（Self-monitoring），(2) 計画との差を評価する自己評価（Self-evaluation），(3) 行動を修正する自己制御（Self-control / adjustment）の三段階の循環過程として説明される。

本研究で提案するシステムは，実績ログ入力による自己モニタリング，SPI や EAC による進捗および将来予測の提示，ならびに「今日のプラン」機能による行動修正支援を通じて，この自己調整プロセス全体を支援する設計となっている。

そこで本章では，提案システムの各機能が，自己調整の各段階に対応した行動変化と関連しているかを明らかにするため，以下の三つの観点から分析を行う。

第一に，SPI を用いた進捗評価により，遅延状態が利用者に認知された際に行動量や作業ペース

に変化が生じるかを分析する（分析 1）。

第二に，EAC による完了予測の提示が，将来的な遅延リスクの認知として機能し，行動の修正や作業量の変化と関連するかを分析する（分析 2）。

第三に,「今日のプラン」機能によって提示されたタスクや作業時間と，実際に選択・実行された行動との対応関係を分析し，行動の選択および時間配分がどのように変化するかを検討する（分析 3）。

これらの分析を通じて，本研究では提案システムが自己調整の各段階においてどのような支援効果を持ちうるかを，定量的ログデータに基づいて明らかにすることを目的とする。

5.2 評価対象および実験条件

本研究では，提案システムを一定期間利用した際に記録されるタスク情報，日次実績ログ，進捗指標，および「今日のプラン」を用いて，自己調整行動への影響を評価することを目的とする。

評価対象

評価対象は，本研究で開発した個人向け進捗管理システムを利用するユーザーであり，主に学習や資格試験対策など，中長期タスクを複数同時に管理する状況を想定する。各ユーザーは，自身でタスクの登録，見積時間の設定，および日次の実績入力を行う。

実験条件

評価期間中，ユーザーは以下の機能を通常利用する。

- タスクの登録および見積時間（E）の設定
- 日次の実績ログ（A）の入力
- pace7d に基づく進捗指標（SPI，EAC）の確認
- 毎日提示される「今日のプラン」に基づく作業

本システムでは，これらの操作により生成されるデータを自動的に記録し，評価に用いる。

収集データ

評価に用いるデータは以下の通りである。

- 各タスクの見積時間，締切日
- 日付ごとの実績作業時間
- 日次で再計算される SPI，EAC，pace7d
- 日付ごとの今日のプラン（提示タスクおよび計画作業時間）

これらのデータを用いて，進捗評価指標が利用者の自己調整行動に与える影響を分析する。

5.3　分析指標および評価方法

本研究では，自己調整学習（self-regulation）を *Self-monitoring*（現状把握），*Self-evaluation*（評価），*Self-control / adjustment*（行動修正）の 3 段階からなる循環的プロセスとして捉える。

提案システムは，実績ログによる現状把握，SPI・EAC による進捗および将来予測の評価，および「今日のプラン」による行動修正支援を提供しており，この自己調整プロセスを包括的に支援する構成となっている。

本節では，これらの機能が利用者の自己調整行動にどのように関与するかを検証するため，以下の 3 つの観点から評価を行う。

- SPI に基づく遅延認知と行動変化（5.3.1）
- EAC に基づく将来予測と危機感の喚起（5.3.2）
- 今日のプランによる行動の選択・配分の変化（5.3.3）

5.3.1　分析 1：SPI に基づく遅延認知と行動変化

SPI（Schedule Performance Index）は，計画に対する進捗効率を表す指標であり，一般に SPI ¡ 1.0 の状態は作業の遅延を示す。本研究では，SPI に基づく進捗評価が利用者に遅延状態を認知させた際，その後の行動にどのような変化が生じるかを分析する。

本システムでは，日次の実績ログに基づいて SPI が更新される。そこで本分析では，SPI が 1.0 を下回った状態で更新が行われたタイミングを**遅延認知イベント**として定義し，当該イベントの前後における進捗指標および行動指標の変化を対象とする。

具体的には，遅延認知イベント直前に算出された SPI（$\mathrm{SPI}_{\mathrm{before}}$）と，実績入力後に再計算された SPI（$\mathrm{SPI}_{\mathrm{after}}$）を比較し，その差分（$\Delta$SPI）を算出する。また，SPI の算出に用いられた評価期間に着目し，週間ペースに基づく評価が継続された場合と，短期的な実績に基づく評価が用いられた場合とで，行動変化の傾向を比較する。

比較は，同一ユーザー内における遅延認知イベント前後の変化を対応付ける前後比較により行う。評価指標としては，SPI の変化量に加え，遅延認知後に SPI が 1.0 以上に回復した割合などを用いる。

また，観測された変化が偶然によるものではないかを確認するため，必要に応じて，同一イベントにおける前後比較に適した対応のある統計的検定を用いて評価する。

以上の分析を通じて，SPI に基づく進捗評価が，利用者における遅延の認知と，その後の行動修正との関連を定量的に明らかにすることを目的とする。

5.3.2　分析 2：EAC に基づく将来予測と行動変化

EAC（Estimate at Completion）は，現在までの実績に基づいてタスクの完了見込み時刻を予測する指標である。本研究では，EAC による将来予測の提示が，利用者に将来的な遅延リスクを認知させ，その後の行動に変化が生じるかを評価する。

具体的には，日次で算出される EAC が，タスクの締切日を超過すると予測された日を**遅延予測イベント**として定義する。各遅延予測イベントについて，その前後一定期間における実績作業時間および作業頻度を算出し，遅延予測の提示前後での行動変化を比較する。

比較は，各ユーザーについて遅延予測イベント前後の行動を対応付ける前後比較により行う。評価指標としては，遅延予測イベント前後における平均作業時間および作業頻度の変化を用いる。

また，これらの変化傾向を定量的に確認するため，必要に応じて，同一ユーザー内での前後比較に適した対応のある統計的検定を補助的に用いる。

以上の方法により，EAC に基づく将来予測の提示が，自己調整学習における Self-evaluation（将来状態の評価）を通じて，Self-control（行動修正）とどのように関連しているかを検証する。

5.3.3 分析 3：今日のプラン遵守度と進捗指標（SPI・EAC）推移との整合性分析

本研究で提案する「今日のプラン」機能は，SPI や EAC といった進捗指標を単に可視化するだけでなく，それらに基づいて当日に取り組むべきタスクと作業時間を具体的な行動計画として提示することを目的としている。本分析では，このプラン提示が利用者の実際の行動選択および時間配分にどの程度反映されているかを検証する。

具体的には，日ごとに生成される「今日のプラン」に含まれるタスクを *planned task*，含まれないタスクを *unplanned task* と定義する。各タスクについて，当日の実績ログを用いて，(1) 実行の有無，および (2) 実際に投入された作業時間を算出する。

まず，planned task と unplanned task の間で，当日に実績が記録された割合（実行率）を比較することにより，今日のプランとして提示されたタスクが，実際の行動選択にどの程度影響を与えているかを評価する。

次に，planned task に着目し，提示された計画作業時間（todayMinutes）に対する実績作業時間の比率を用いて，**プラン遵守度（plan adherence）** を算出する。これにより，単にタスクが実行されたか否かだけでなく，提示された作業量がどの程度実行されたかという，時間配分の観点から行動を評価する。

さらに，プラン遵守度と，その翌日以降の SPI の変化量（ΔSPI），および EAC による予測完了日の変化量との関係を分析する。これにより,「今日のプラン」に沿った行動が，進捗効率の改善や締切超過リスクの低減とどのように関連しているかを検証する。

本分析は，SPI や EAC といった進捗指標そのものを直接操作対象とするものではなく，それらを基に生成された「今日のプラン」を介して，利用者が間接的に進捗指標を考慮した行動を取れているかを評価することに主眼を置く。

以上により,「今日のプラン」機能が，自己調整学習における Self-control（行動修正）を具体化する支援として機能しているかを明らかにする。

6 結果・考察

6.1 分析 1：週間評価フェーズにおける SPI 改善効果の検証

6.1.1 使用データの概要

本分析では，提案システムの実利用ログとして取得された日次作業記録データ（`daily_logs.csv`）を用いた。本データには，各タスクに対する日付ごとの実績作業時間および推定作業時間などが含まれている。

分析に先立ち，列名の統一および型変換を行った。具体的には，タスク識別子を `taskId` に統一し，日付情報（`date`，`dueDate`）を日付型へ変換した。また，実績作業時間（`minutes`）および推

表 4: 日次作業ログにおける記述統計量

	minutes	estimateMinutes
count	186.000	186.000
mean	132.844	6148.726
std	136.431	6823.008
min	0.000	3.000
25%	30.000	90.000
50%	90.000	1440.000
75%	180.000	15000.000
max	1000.000	15000.000

定作業時間（`estimateMinutes`）については数値型として扱い，欠損値は 0 として補完した。その後，タスク単位・日付順にデータを整列した。

前処理後のデータは，全体で 186 行の日次ログから構成され，対象タスク数は 48 件であった。本分析では，この日次ログを基に，週間評価フェーズにおける SPI の変化と利用者行動の関係を検証する。

6.1.2 日次作業量および推定作業量の分布

Table 4 に，実績作業時間（`minutes`）および推定作業時間（`estimateMinutes`）の記述統計量を示す。

実績作業時間の中央値は 90 分，第 1 四分位は 30 分，第 3 四分位は 180 分であり，日次の作業量には一定のばらつきが確認された。一方で，最大値は 1000 分であり，一部の日に集中的な作業が行われていることが分かる。

推定作業時間は，タスク全体に対する総見積もり時間を表しており，中央値は 1440 分であった。また，第 3 四分位および最大値はいずれも 15000 分であり，本研究で扱うタスクには，短期間で完結するものから長期的な大規模タスクまでが含まれていることが示唆される。

以上より，本分析に用いるデータは，日次単位での行動変化を捉えるのに十分な粒度を持ち，かつ多様なタスク規模を含む実利用ログであることが確認された。

6.1.3 SPI 算出に用いる指標の定義

本分析では，SPI（Schedule Performance Index）を「当日朝時点で要求される作業ペース（required）」と「直近の実績に基づく作業ペース（pace）」の比として定義する。

必要作業ペース（required）　各タスクについて，推定作業時間を E，当日朝時点までの累積実績作業時間を C_{t-1}，締切日までの残日数を D_t とすると，当日朝時点における残作業量 R_t は次式で表される。

$$R_t = \max(0, E - C_{t-1})$$

これより，当日朝時点で要求される 1 日あたりの作業量（required）は，

$$\text{required}_t = \begin{cases} \dfrac{R_t}{D_t} & (R_t > 0) \\ 0 & (R_t = 0) \end{cases}$$

と定義する。なお，締切日当日も含めて評価するため，D_t は最小値を 1 日とした。

直近実績ペース（pace7d）　利用者の直近の行動を反映するため，直近 7 日間の実績作業時間を用いた平均作業ペース pace7d を算出する。

ただし，実利用ログにおいては，連続的な作業が行われない場合も多く，単純に 7 日で割ると実態を過小評価する恐れがある。そこで，本研究では以下のルールを導入した。

- 直近 7 日間の実績作業時間の合計を S_7 とする（作業がない日は 0 とする）
- 実績が存在する日数を N とする
- $N < 3$ の場合は短期評価フェーズとみなし，分母を N とする
- $N \geq 3$ の場合は週間評価フェーズとみなし，分母を 7 とする

以上より，pace7d は次式で定義される。

$$\text{pace7d} = \frac{S_7}{\max(1,\ N < 3 ? N : 7)}$$

また，$N < 3$ の場合を**短期評価フェーズ（Short Window）**，$N \geq 3$ の場合を**週間評価フェーズ**として区別した。

SPI の定義　当日朝時点（before）および当日実績入力後（after）について，SPI を次式で算出する。

$$\text{SPI} = \begin{cases} \dfrac{\text{pace7d}}{\text{required}} & (\text{required} > 0) \\ 1.0 & (\text{remaining} = 0) \\ 0.0 & (\text{remaining} > 0 \wedge \text{required} = 0) \end{cases}$$

これにより，

- $\text{SPI} < 1$ は計画に対して遅延傾向にある状態
- $\text{SPI} = 1$ は計画通りの進捗
- $\text{SPI} > 1$ は計画を上回る進捗

を表す。

本分析では，当日朝時点の $\text{SPI}_{\text{before}}$ と当日実績入力後の $\text{SPI}_{\text{after}}$ の差分

$$\Delta\text{SPI} = \text{SPI}_{\text{after}} - \text{SPI}_{\text{before}}$$

を用いて，実績入力が SPI に与える即時的な影響を評価する。

表 5: 入力イベントの評価モード遷移ごとの件数

モード遷移（before→after）	件数
weekly→weekly	92
short→short	65
short→weekly	14

6.1.4　分析 1：週ベース評価フェーズにおける SPI 改善効果の検証（結果）

本分析では，日次ログから算出した $\mathrm{SPI}_{\mathrm{before}}$ と $\mathrm{SPI}_{\mathrm{after}}$ の差分 $\Delta\mathrm{SPI} = \mathrm{SPI}_{\mathrm{after}} - \mathrm{SPI}_{\mathrm{before}}$ を用いて，実績入力（行動）が SPI を改善する方向に作用しているかを検証した。

更新イベントの抽出　本研究では，実績入力により SPI が変化した日を「入力イベント」とみなし，$\Delta\mathrm{SPI} \neq 0$ を満たす行のみを抽出した。その結果，日次ログ全 186 行のうち 171 行が入力イベントとして抽出された。

評価モードの分類（短期評価／週ベース評価）　SPI の分母側で用いる作業ペース pace7d は，直近 7 日間の実績日数が 3 日未満の場合，短期評価フェーズ（Short Window）として算出ルールが変化する（分母が 7 ではなく実績日数となる）。この算出モードの違いは SPI のスケール解釈に影響するため，入力イベントごとに当日朝時点（before）と入力後（after）それぞれの評価モードをラベル化し，`weekly`（週ベース）／`short`（短期）として区別した。

モード遷移に基づくケース分割　さらに，before/after の組合せにより，入力イベントを以下の 3 群に分類した。

- **weekly→weekly**：週ベース同士で比較可能なイベント
- **short→short**：短期評価同士のイベント
- **short→weekly**：短期から週ベースへ切り替わったイベント（遷移）

分類結果を表 5 に示す。本研究の主分析では，尺度の一貫性が保たれる weekly→weekly を主対象とし，short→short および遷移ケースは参考値として扱う。

weekly→weekly イベントの基本統計　weekly→weekly（$n = 92$）における主要変数の基本統計を表 6 に示す。ΔSPI は平均 0.273，中央値 0.256 であり，入力後に SPI が増加する方向に偏る傾向が確認された。また，pace7d は平均で 150.96（before）から 177.04（after）へ増加しており，入力当日の実績が直近ペースを押し上げたことが分かる。

短期評価・遷移ケースの扱い　short→short では ΔSPI の分散が大きく，極端な値（例：最大 69.6）が出現した。これは短期評価フェーズにおいて分母（平均化に用いる日数）が小さくなり得るため，pace7d および SPI が不安定になりやすいことに起因すると考えられる。また short→weekly は評価モード自体が変化するため，before/after の差分が純粋な行動効果だけでなく，算出ルールの変更も含む可能性がある。以上より，本章では weekly→weekly を主対象として検定を行い，短期評価および遷移ケースは補助的に解釈する。

表 6: weekly→weekly イベント（$n = 92$）の基本統計

変数	平均	標準偏差	最小	中央値	最大
ΔSPI	0.273	0.202	0.0049	0.256	1.429
$\mathrm{SPI}_{\mathrm{before}}$	1.506	0.653	0.0286	1.570	3.523
$\mathrm{SPI}_{\mathrm{after}}$	1.779	0.761	0.0357	1.833	3.811
required	145.61	207.20	3.50	93.82	1200.00
$\mathrm{pace7d}_{\mathrm{before}}$	150.96	73.64	7.86	160.71	300.00
$\mathrm{pace7d}_{\mathrm{after}}$	177.04	83.84	12.86	188.57	351.43

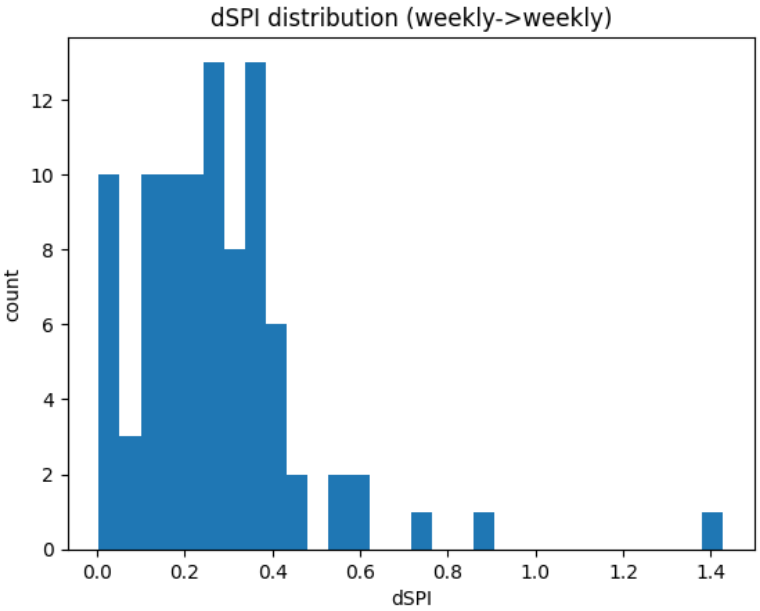

図 10: weekly→weekly イベントにおける ΔSPI の分布

6.1.5 統計検定による SPI 変化の検証

前節で抽出した入力イベントに対し，実績入力に伴う SPI の変化量 $\Delta SPI = SPI_{\mathrm{after}} - SPI_{\mathrm{before}}$ が正の方向に偏っているかを検証するため，ノンパラメトリック検定を用いた統計的評価を行った。

本分析では，評価尺度の一貫性が保たれる weekly→weekly イベントを主な分析対象とした。ΔSPI は分布の非対称性および外れ値を含む可能性が高く，正規分布を仮定することが困難であるため，中央値が 0 より大きいかを検定する Wilcoxon の符号付順位検定（片側検定）を用いた。

その結果，weekly→weekly イベント（$n = 92$）において，ΔSPI は有意に正の方向へ偏っており（$p < 0.001$，効果量 $r = 0.87$），入力後に SPI が改善方向へ変化する事例が多く含まれていることが示された。ΔSPI の中央値は 0.256，平均値は 0.273 であり，入力イベントが SPI の上昇と関連している可能性が示唆される。

図 10 に，weekly→weekly イベントにおける ΔSPI の分布を示す。分布は 0 以上の値に集中しており，入力イベント後に SPI が改善する傾向が視覚的にも確認できる。

また，SPI の入力前後の関係を図 11 に示す。多くの点が $SPI_{\mathrm{after}} > SPI_{\mathrm{before}}$ の領域に分布し

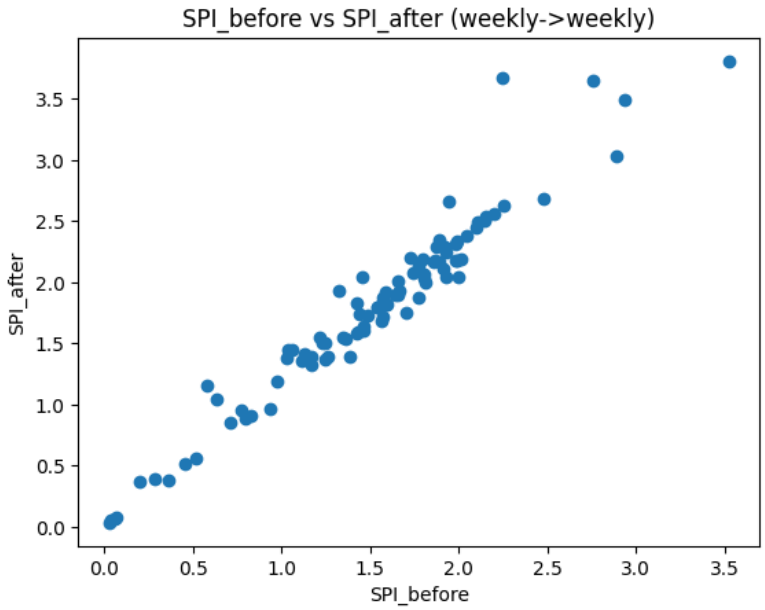

図 11: weekly→weekly イベントにおける SPI の入力前後比較

ており，実績入力後に進捗評価が改善する傾向が確認できる。

一方，短期評価が継続した short→short イベント（$n = 65$）では，ΔSPI の中央値は 1.0，平均値は 4.41 と正の値を示したものの，分散が大きく，極端な値（最大 69.6）も観測された。図 12 に示すように，分布は強い歪みを持ち，SPI が不安定に変動している様子が確認できる。

これは，短期評価フェーズでは平均化に用いられる日数が少なく，pace7d および SPI が入力 1 回の影響を強く受けるため，指標が不安定になりやすいことに起因すると考えられる。また，short→weekly のような評価モード遷移を伴うイベントでは，評価基準自体が変化するため，ΔSPI は純粋な行動変化だけでなく，算出ルールの影響も含む可能性がある。

以上より，本分析では評価尺度の一貫性が保たれる weekly→weekly イベントを主対象として解釈することが妥当であると判断した。週ベース評価フェーズにおいては，実績入力が SPI の改善と統計的に有意な関連を持つことが示され，SPI に基づく進捗提示が，自己調整学習における Self-evaluation（進捗の評価）から Self-control（行動修正）への移行を一定程度支援している可能性が示唆される。

6.1.6 $\mathbf{SPI_{after}} \geq 1$ に到達したイベントの特徴

実績入力後に $SPI_{\text{after}} \geq 1$ を満たしたイベント（進捗指標が少なくとも遅延状態から脱したとみなせるケース）を抽出し，評価モード遷移別に SPI_{before}, SPI_{after}, および ΔSPI（$= SPI_{\text{after}} - SPI_{\text{before}}$）の分布を要約した（表 7）。

週ベース評価が維持された weekly→weekly（$n = 78$）では，SPI_{before} の平均 1.699 から SPI_{after} の平均 2.008 へ上昇し，ΔSPI も平均 0.309（中央値 0.274）と正の増加が観察された。これは，週ベース評価フェーズにおいては，実績入力が指標の改善（少なくとも $SPI \geq 1$ を満たす方向）と整合的に現れるケースが多いことを示す。

一方，short→short（$n = 46$）では，SPI_{after} および ΔSPI が極端に大きい値（最大 69.6）を

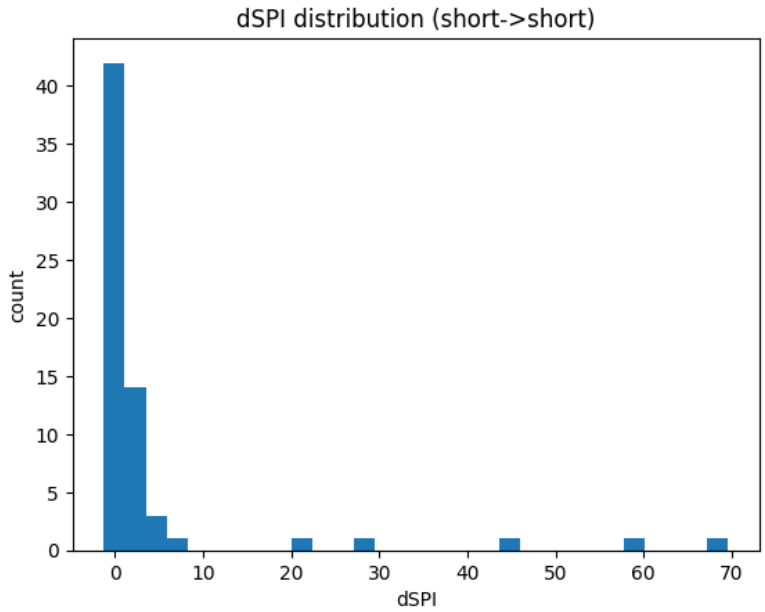

図 12: short→short イベントにおける ΔSPI の分布

表 7: $SPI_{\text{after}} \geq 1$ を満たすイベントの記述統計（モード遷移別）

モード遷移	n	SPI_{before} 平均	SPI_{after} 平均	ΔSPI 平均	ΔSPI 中央値	ΔSPI 最大
weekly→weekly	78	1.699	2.008	0.309	0.274	1.429
short→short	46	0.635	6.706	6.071	1.130	69.600
short→weekly	4	2.564	1.340	-1.225	-0.988	-0.179

取り得た。短期評価は平均化窓が小さくなりやすく，required や pace7d の比が増幅されることで SPI が不安定になり，外れ値が発生しやすい性質を反映していると考えられる。したがって，本研究の主要な解釈は評価尺度の一貫性が保たれる weekly→weekly に限定し，short→short は参考として扱う。

また，short→weekly（$n = 4$）では ΔSPI が負の値に偏った。これは入力効果というよりも，評価モードの切替に伴う算出ルールの変化が支配的となり，before/after の差分解釈が困難になるケースである。

6.1.7 入力後に $SPI_{\text{after}} \geq 1$ を満たす割合

各入力イベントについて，入力後の進捗指標が $SPI_{\text{after}} \geq 1$ を満たした割合（到達率）を，評価モード遷移別に算出した（表 8）。ここで $SPI_{\text{after}} \geq 1$ は，入力後に少なくとも遅延状態から脱している（要求ペース以上に到達している）状態として解釈できる。

週ベース評価が維持された weekly→weekly では到達率が 0.8478 と最も高く，実績入力後に進捗評価が改善（少なくとも $SPI \geq 1$ を満たす）するケースが多いことが示された。一方，短期評価 short→short は 0.7077 であり，到達率自体は一定程度高いものの，前節で述べた通り指標が不安定になりやすく外れ値を含むため，解釈は参考に留める。

表 8: モード遷移別の $SPI_{\text{after}} \geq 1$ 到達率

モード遷移	到達率（$SPI_{\text{after}} \geq 1$）
weekly→weekly	0.8478
short→short	0.7077
short→weekly	0.2857

また short→weekly は 0.2857 と低く，評価モードの切替が介在する場合には，入力前後差が行動変化ではなく算出ルールの変化に影響されやすいことが示唆される。以上より，本研究の主要な評価は尺度の一貫性が保たれる weekly→weekly を中心に行うことが妥当である。

本分析より、週ベース評価フェーズにおいては、実績入力に基づく SPI の提示が、進捗評価の改善と統計的に有意な関連を持つことが示された。これは、SPI による進捗の可視化が、自己調整学習における Self-evaluation から Self-control への移行を一定程度支援している可能性を示唆する。

6.2 分析 2：EAC に基づく将来予測と行動変化

6.2.1 分析の目的

本分析では，将来予測指標である EAC（Estimate at Completion）が，タスクの締切超過を示唆した際に，利用者の行動（作業量）に変化が生じているかを検証する。

EAC は，現在までの実績に基づいてタスクの完了見込み時刻を予測する指標であり，SPI のような即時的な進捗評価とは異なり，将来的な遅延リスクを可視化する役割を持つ。本研究では，EAC による将来予測の提示が，利用者に遅延リスクを認知させ，自己調整学習における行動修正（Self-control）とどのように関連するかを評価する。

6.2.2 使用データおよび遅延予測イベントの定義

日次で算出される EAC が，タスクの締切日を超過すると予測された日を，**遅延予測イベント**として定義した。

具体的には，EAC の予測完了日（`eacDateKey`）が締切日（`deadlineKey`）を超過している日を遅延予測イベントとして抽出した。本分析では，この条件を満たすイベントのみを対象とし，その結果，全イベントのうち 20 件が遅延予測イベントとして抽出された。

6.2.3 遅延予測イベント前後の作業量変化

各遅延予測イベントについて，イベント発生日の直前 7 日間および直後 7 日間における 1 日あたり平均作業時間を算出した。

表 9 に，遅延予測イベント前後の平均作業時間および差分の記述統計量を示す。

図 13 に，遅延予測イベント前後の平均作業時間の分布を箱ひげ図として示す。多くのケースにおいて，遅延予測イベント後の作業量は，イベント前よりも小さくなっていることが視覚的に確認できる。

表 9: 遅延予測イベント前後の作業時間の記述統計（$n = 20$）

	平均	標準偏差	最小	中央値	最大
前（before）	31.81	28.58	2.14	21.43	85.71
後（after）	16.19	25.18	0.00	2.93	88.57
差分（after–before）	-15.62	23.59	-60.00	-7.86	28.57

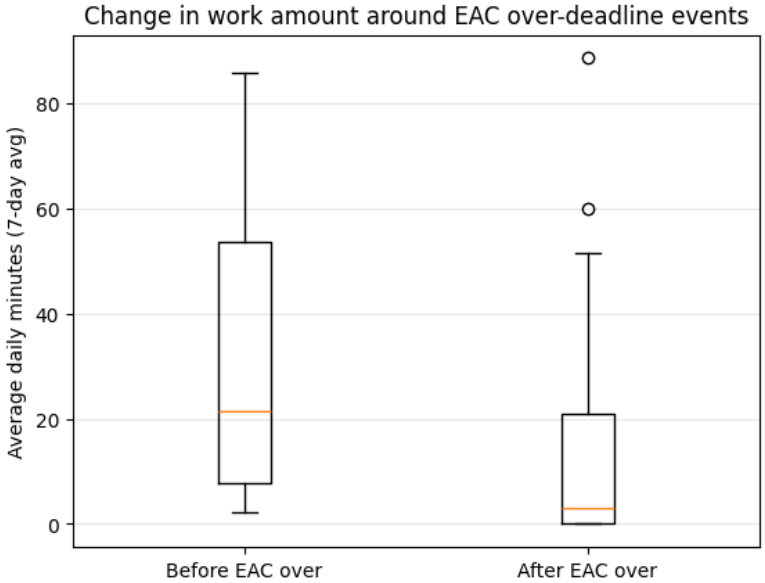

図 13: 遅延予測イベント前後の平均作業時間の分布

6.2.4　統計検定による前後差の検証

遅延予測イベント前後の作業時間差について，差分データの分布特性を確認するため，正規性の検定として Shapiro–Wilk 検定を行った。その結果，差分は正規分布からの有意な逸脱は認められなかった（$p = 0.27$）。

そこで，対応のある t 検定を用いて前後の平均作業時間の差を検証した。その結果，遅延予測イベント後の作業時間は，イベント前と比較して有意に減少していた（$t(19) = 2.96, p < 0.01$）。

また，対応のある t 検定に基づく効果量として算出した r は 0.66 であり，中程度から大きい効果が示された。さらに，遅延予測イベント後に作業時間が増加した割合は 0.15 であり，全体の 85% のイベントにおいて，作業量は維持または減少していた。

6.2.5　結果の解釈と考察

以上の結果より，EAC によって将来的な締切超過が示唆された場合，多くの利用者において，直後の作業量が増加するのではなく，むしろ減少する傾向が観察された。

これは，EAC が「行動を即時に修正させる警告」として機能したというよりも，将来予測を通じた Self-evaluation （将来的な進捗状態の再評価）を促した結果，作業計画の見直しや一時的な行動抑制が生じた可能性を示唆する。

この点で，SPI が短期的な行動修正（Self-control）を直接的に促す指標であるのに対し，EAC は中長期的な見通しの再評価を支援する指標として，自己調整学習における役割が異なることが示唆される。

7　まとめ

参考文献

[1] 野上 俊一, 生田 淳一, 丸野 俊一,“テスト勉強の学習計画と実際の学習活動とのズレに対する認識”, 日本教育工学会論文誌, 28(suppl), pp. 173–176, 2005.

[2] Project Management Institute, “A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition”, Project Management Institute, 発行年:2021.

[3] NASA, “Earned Value Management (EVM) Implementation Handbook”, NASA, 発行年:2019.

[4] U.S. Department of Defense, “Earned Value Management System (EVMS) Guidelines”, DoD, 発行年:2018.

A　当日の作業計画におけるタスク選定アルゴリズム

本付録では，第 4 章で概要を述べた「今日のプラン」機能において用いられる，当日の作業計画（最大 3 件）を生成するアルゴリズムの実装仕様を示す。本アルゴリズムは，進捗指標および締切情報を統合的に参照し，候補タスクの優先度評価と日次キャパシティに基づく作業分数の配分を行う。

アルゴリズムはフロントエンドおよびバックエンドの双方から利用可能な共通モジュール（`shared/planEngine.js`）として実装されており，実行環境に依存しない一貫した判定結果が得られる構成としている。

A.1　アルゴリズムの入力と出力

入力は，各タスク i に対して以下の情報である。

- 見積作業時間 E_i，累積実績時間 A_i，残作業量 $R_i = E_i - A_i$
- 締切日時 $deadline_i$
- 予定開始日時 $plannedStart_i$
- 必要ペース（`requiredPaceAdj` または `requiredPace`）
- 進捗率（`idealProgress`，`actualProgress` など）
- ユーザー設定：日次キャパシティ C（`dailyCap`）

出力は，当日に取り組むタスク集合 $\mathcal{P}$ と各タスクへの割当分数（`todayMinutes`）である。本実装ではキャパシティ制約により割当が 0 分となるタスクは除外されるため，$|\mathcal{P}|$ は最大 3 件であるが，状況により 3 件未満となる場合がある。

補足（実装上の呼出し形態）　本モジュールは再利用性のため，呼び出し側から一時的なキャパシティ指定や残キャパシティを受け取れる設計としている。これらは内部引数（例：`capOverride`，`remainingCap`，`mode`）として扱われるが，本付録では主要な入出力を明確にするため，詳細は割愛する。

A.2　日次キャパシティの決定

当日に利用可能な作業時間の上限を日次キャパシティ C［分］とする。C が未設定または $C \leq 0$ の場合には既定値として $C = 120$ 分を用いる。また計算過程で C が不正値となった場合にも，安全策として既定値にフォールバックする。

A.3　候補タスク集合の定義

各タスク i について，以下の条件をすべて満たすものを候補タスク集合 $\mathcal{S}$ に含める。

- 予定開始日時 $plannedStart_i$ が未来でない（$plannedStart_i \leq now$）
- 見積作業時間 $E_i > 0$

- 累積実績 $A_i < E_i$（未完了）
- 締切日時 $deadline_i$ が設定され，日付として解釈可能である

A.4　遅れ度および残日数の定義

候補タスク $i \in \mathcal{S}$ について，理想進捗率および実進捗率を

$$\mathrm{ideal}_i = \frac{t_i}{T_i}, \qquad \mathrm{actual}_i = \frac{A_i}{E_i}$$

で定義し，遅れ度 lag_i は

$$\mathrm{lag}_i = \mathrm{ideal}_i - \mathrm{actual}_i$$

とする。進捗超過（$\mathrm{lag}_i < 0$）はスコア計算において 0 とみなす。
締切までの残日数 D_i は実装上,

$$D_i = \max\left(1, \left\lceil \frac{deadline_i - now}{1\,day} \right\rceil\right)$$

として整数日で扱う（当日締切の場合も $D_i = 1$ となる）。

A.5　優先度スコアの算出

候補タスク i の優先度は，遅れ度，締切リスク，残作業負荷を統合したスコア S_i により評価する。

$$S_i = 3 \cdot \max(0, \mathrm{lag}_i) + 2 \cdot \frac{1}{D_i + 1} + \frac{R_i}{D_i}$$

候補集合 $\mathcal{S}$ を S_i の降順で整列する。
同点の場合は「締切が近い順」→「必要ペース（`requiredPaceAdj` または `requiredPace`）が大きい順」で決定する。

A.6　日次キャパシティに基づく割当（`todayMinutes`）

本実装では，優先度上位タスクを単に最大 3 件返すのではなく，日次キャパシティ C の範囲内で各タスクに当日作業分数（`todayMinutes`）を配分する。配分は概ね以下の段階で行い，割当が 0 分となるタスクは当日の計画対象から除外される。

1. **最低作業量（required）の割当**：必要ペース（`requiredPaceAdj`/`requiredPace`）を基準に，各タスクへ最低限の分数を割り当てる。
2. **回復目標（recover）の割当**：進捗遅れ（lag_i）やリスクガイダンスに基づいて算出される回復量（`recoverMinutes`）を目標とし，キャパシティが許す範囲で追加配分する。
3. **余剰（surplus）の配分**：上記を満たした後にキャパシティが残る場合，締切が近い順に，残作業量に基づき最大 30 分を上限として追加配分する。

この結果，キャパシティが小さい場合には，最大 3 件未満のタスクのみが計画に残る。

A.7 フォールバック処理

割当の結果，当日の作業計画 $\mathcal{P}$ が空となった場合に限りフォールバック処理を行う。フォールバックでは，候補集合 $\mathcal{S}$ の並び（優先度スコアに基づく整列結果）をそのまま用いて最大 3 件を対象とし，締切までの残日数に基づく日割り負荷を 5 分単位で切り上げて割り当てる。候補集合 $\mathcal{S}$ 自体が空である場合は，フォールバックでも計画は空となる。

A.8 疑似コード

```
C = resolveDailyCap(dailyCap)

S = extractCandidateTasks(
  plannedStart <= now,
  E > 0,
  A < E,
  valid deadline
)

for task in S:
  compute lag, D, R
  compute score S_i

sort S by (S_i desc, deadline asc, requiredPace desc)

P = allocateTodayMinutes(S, C)   # required -> recover -> surplus
remove tasks with todayMinutes == 0

if P is empty:
  if S not empty:
    P = fallbackAllocateTop3(S)  # uses sorted S order
  else:
    P = empty

return P
```